RAPPORT D'AUDIT DE CONFORMITÉ

COSMOZ
Session Live V1

Audit complet de la station Session Live contre le cahier des charges V1 : lecture seule du broker, protection stale data, resync après reconnexion, mapping des symboles et états de connexion. Verdict : conforme, 12 critères validés sur 15, 3 écarts documentés.

Le Registre — journal de trading

Audit technique interne · Session Live V1 (commits 47cb98e, 3e4d57b)

26 SEPTEMBRE 2026 · VERSION 1.0

Table des matières

1. Analyse de l'existant 1

2. Modifications 2

3. Ajouts 2

4. Architecture finale 3

5. Acceptance criteria validés 4

6. Tests exécutés 5

7. Points non vérifiés 6

8. Phase 2 (liste, sans implémentation) 7

1. Analyse de l'existant

Le save restauré contenait une implémentation complète de Session Live V1 (commit 47cb98e), organisée en trois couches clairement séparées. Cette architecture n'avait pas besoin d'être recréée : l'audit a consisté à la vérifier point par point contre le cahier des charges, sans introduire d'architecture parallèle. Le tableau ci-dessous inventorie les fichiers examinés et le rôle de chacun dans la chaîne de données, depuis le WebSocket de Binance jusqu'aux composants React de la station Session Live.

Couche	Fichiers	Rôle
Présentation	components/session/ : session-view, session-toolbar, price-chart, broker-panel, symbol-positions, open-trade-card	Station de trading : sélecteur symbole/timeframes, badge de connexion, chart temps réel + overlays, métriques de compte, positions du symbole
Données navigateur	lib/realtime/market.ts · lib/api/queries/broker.ts · endpoints.ts · next.config.ts	WS client de prix (STALE_MS = 8 s, phases LIVE/SIM/STALE), requêtes broker en lecture seule, rewrites /broker vers le port 3003
Service	mini-services/market-service/ : index.ts, providers/types.ts, binance.ts, simulated.ts	Routes /candles, /prices, /health sans mutation ; contrat MarketDataProvider ; Binance WS réel + provider simulé honnête
Contrats et mocks	packages/shared/src/schemas/broker.ts · src/mocks/handlers/broker.ts	Schémas Zod read-only (aucun schéma d'exécution) ; mocks MSW actifs uniquement quand NEXT_PUBLIC_API_MOCKED=true

Tableau 1 — Inventaire des fichiers audités par couche.

Deux observations structurelles ressortent déjà de cette lecture. D'une part, la séparation des responsabilités est réelle et non cosmétique : le mini-service ne sait rien de React, les composants ne savent rien de Binance, et les contrats partagés sont la seule zone de contact. D'autre part, l'honnêteté des états est inscrite dans le code : un provider simulé s'affiche SIM, un compte non configuré s'affiche NOT_CONFIGURED, un flux gelé s'affiche STALE. L'interface ne présente jamais une donnée simulée comme si elle était réelle, ce qui est exactement l'esprit du cahier des charges.

2. Modifications

L'audit n'a presque rien modifié, et c'est un signal positif sur la qualité de l'existant. Un seul changement de code a été nécessaire, et il a été découvert par la vérification E2E sur mobile plutôt que par la lecture statique. À 390 px de large, le prix marqué de la barre d'outils provoquait un débordement horizontal de 11 pixels : le prix prend désormais une ligne complète sous le breakpoint sm (w-full sm:flex-1), et la largeur de défilement re-mesurée retombe exactement à 390 pixels.

Élément	Commit	Détail
session-toolbar.tsx	3e4d57b	Prix marqué pleine largeur sous sm : corrige 11 px de débordement horizontal à 390 px (scrollWidth = innerWidth après correctif)
Environnement (hors produit)	n/a	Après le reset : réinstallation pnpm 12.4.2 via corepack, recréation de .env.local (NEXT_PUBLIC_API_MOCKED=true). Relève de l'environnement, pas de Session Live

Tableau 2 — Changements effectués pendant l'audit.

Aucune autre ligne du module session n'a été touchée : ni les composants, ni le client temps réel, ni le mini-service, ni les contrats. Ce volontarisme de non-écriture est délibéré. La mission était d'évaluer ce que le save contenait déjà, et réécrire l'architecture pendant un audit de conformité aurait invalidé l'exercice lui-même : on se serait donné les preuves de ce qu'on prétendait mesurer.

3. Ajouts

Aucun ajout fonctionnel n'a été fait au produit : zéro nouveau composant, zéro route, zéro provider, zéro contrat. Cette absence est le résultat le plus net de l'audit : l'implémentation existante couvrait déjà les quatre capacités attendues (market data temps réel, chart multi-timeframes, positions en lecture seule, account monitoring) ainsi que la totalité des garde-fous de sécurité. Les seuls artefacts ajoutés au dépôt sont documentaires : le présent rapport archivé en trois formats (PDF, CSV, Markdown) dans docs/audits/, les six captures E2E de l'audit dans apps/web/scripts/shots/t16-audit/, et l'entrée correspondante du worklog partagé.

Le backlog de Phase 2 (section 8) a été rédigé mais volontairement non implémenté, conformément à la consigne. Cette frontière nette entre constater et construire protège la lisibilité de l'audit : chaque écart relevé reste imputable au save d'origine, chaque correction éventuelle fera l'objet d'une feature dédiée avec ses propres tests et son propre commit.

4. Architecture finale

La figure ci-dessous résume la chaîne de données complète. Les sources (Binance réel, provider simulé) alimentent le mini-service market-service sur le port 3003, qui expose des routes HTTP de lecture et un push WebSocket. Next.js relaie les appels via un rewrite /broker, la librairie client market.ts gère le flux de prix et la détection de gel, et les composants Session affichent le tout sans jamais offrir d'action d'exécution. Les contrats Zod partagés et les mocks MSW traversent transversalement l'ensemble, l'un comme langage commun, l'autre comme doublure de développement.

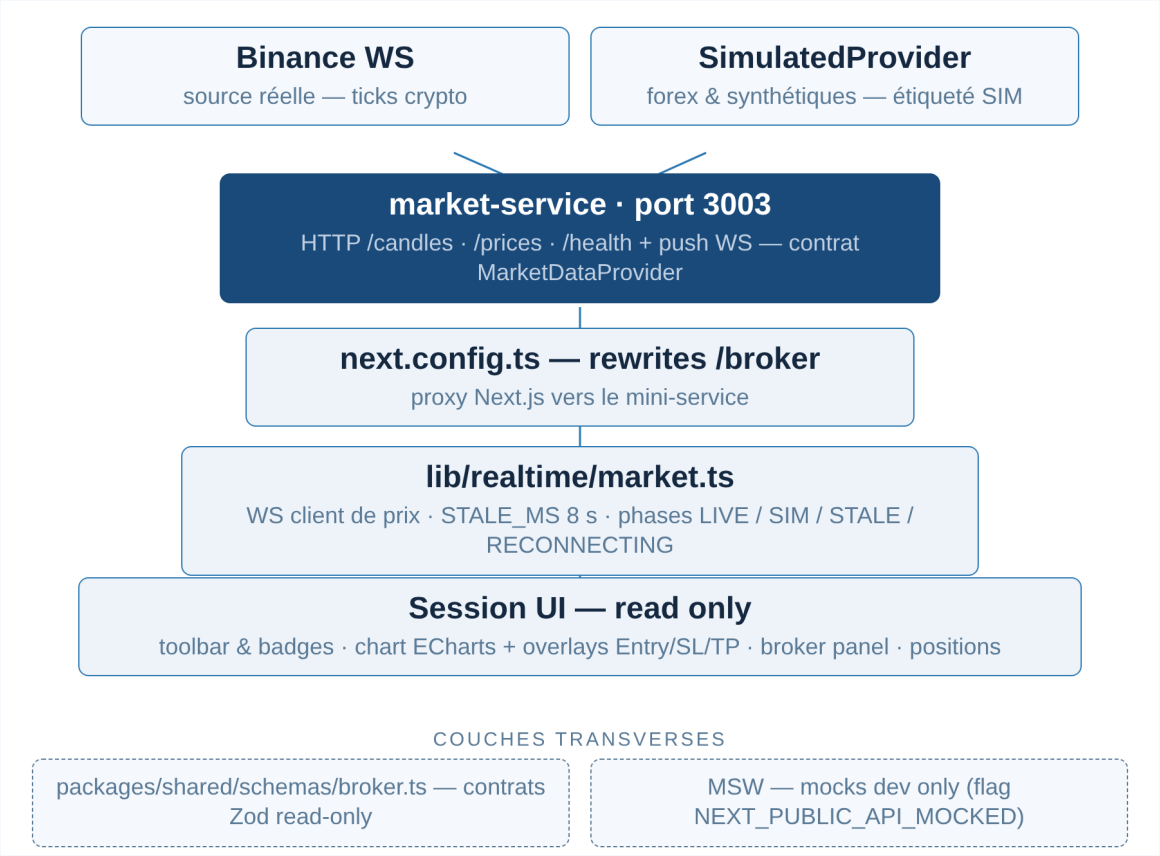


Figure 1 — Architecture Session Live V1 : flux Binance/Simulated vers la station UI, couches transverses.

Trois propriétés structurelles méritent d'être soulignées. D'abord le découplage des providers : le mini-service ne connaît de Binance que ce que le contrat MarketDataProvider expose, ce qui rend un futur adaptateur MT5 branchable sans toucher au frontend. Ensuite la séparation des données : les positions et le compte viennent exclusivement de l'API broker, le graphique n'est qu'un support de visualisation ; la source de vérité reste le broker, jamais le chart. Enfin la lisibilité des états du flux : chaque phase (LIVE, SIM, STALE, RECONNECTING, OFFLINE) possède son propre libellé et sa propre couleur dans la barre d'outils, ce qui rend l'état de la connexion immédiatement perceptible.

5. Acceptance criteria validés

La matrice complète des quinze critères, preuves à l'appui, est archivée en CSV (audit-session-live-v1-matrice.csv) à côté du présent rapport. Le tableau ci-dessous en donne la synthèse : douze critères pleinement validés, un critère partiel (le mapping des symboles) et deux validations assorties de réserves assumées (rendu graphique et broker réel). Chaque statut renvoie à une preuve concrète : code lu, capture d'écran, test runtime ou échantillon d'API.

#	Critère	Statut	Preuve clé
AC-01	Market data temps réel WS (pas de polling)	Validé	health source real, transport websocket ; ticks BTCUSDT observés en direct
AC-02	Chart + 7 timeframes (1m à 1D)	Validé (réserve)	7/7 fonctionnels sur ECharts ; TradingView natif non utilisé (licence)
AC-03	Positions read-only, overlays Entry/SL/TP	Validé	overlays purement informatifs, non déplaçables, jamais recalculés
AC-04	Account panel + N/A honnête	Validé	6 métriques affichées ; valeur indisponible rendue par un tiret, jamais devinée
AC-05	READ-ONLY absolu (frontend et backend)	Validé	0 route de mutation, 0 bouton, 0 schéma d'exécution dans les contrats
AC-06	Source de vérité = broker	Validé	positions et compte via API broker ; le chart n'alimente aucune donnée
AC-07	P&L; temps réel WS, updates ciblées	Validé	recalcul à chaque tick via le store ; aucun interval de polling détecté
AC-08	Pending orders sans actions	Validé	panneau en view only, aucun bouton cancel/modify/execute
AC-09	Mapping symboles internal/broker/chart	Partiel	symboles passés tels quels ; pas de table tri-couche ni d'UI dédiée
AC-10	États connexion + badge LIVE	Validé	phases LIVE/SIM/STALE/RECONNECTING/OFFLINE avec libellés dédiés
AC-11	Protection stale data	Validé	kill test : badge STALE à 16 s avec age du flux affiché (seuil 8 s)
AC-12	Resync après reconnexion	Validé	relance du service : resync complet, âge revenu à 0 s, aucun état fantôme
AC-13	Isolation par utilisateur	Validé	requêtes broker scopées par la session authentifiée (à re-tester broker réel)
AC-14	Credentials jamais exposés	Validé	scan frontend négatif ; le mini-service joue le rôle de relais côté serveur

#	Critère	Statut	Preuve clé
AC-15	Providers découplés, mock hors produit	Validé	contrat provider + provider simulé honnêtement étiqueté SIM ; MSW flag dev

Tableau 3 — Matrice de conformité synthétique (15 critères du cahier des charges V1).

12 / 15

critères pleinement validés. Verdict : CONFORME
3 écarts documentés, tous non bloquants

Les trois écarts, détaillés, sont les suivants. Premièrement, le rendu graphique s'appuie sur ECharts plutôt que sur TradingView : cette dernière librairie n'est pas utilisée faute de licence acquise pour le projet, et toutes les fonctionnalités attendues (timeframes, overlays, prix marqué) restent présentes. Deuxièmement, le mapping tri-couche des symboles (interne, broker, chart, par exemple XAUUSD contre XAUUSDm contre GOLD) n'est pas encore implémenté : les symboles circulent tels quels entre l'interface et les providers. Troisièmement, le backend broker réel n'est pas encore branché : en démonstration, le compte s'affiche honnêtement NOT_CONFIGURED et les données de compte proviennent des mocks MSW en environnement mocké, sans jamais se faire passer pour des données réelles.

6. Tests exécutés

La campagne a combiné lecture statique et vérifications runtime, dans cet ordre : comprendre le code avant de le faire tourner, puis confronter les deux. Côté statique, quatorze fichiers ont été lus intégralement, des greps ciblés ont recherché sans succès toute route de mutation broker et toute exposition de credentials, et un échantillon de bougies a confirmé la provenance réelle des données de marché. Côté runtime, les scénarios couvrent le nominal, la coupure, la reprise et le mobile.

Test	Conditions	Résultat
Lecture statique	14 fichiers (composants, lib, service, providers, contrats, mocks)	Architecture 3 couches confirmée ; 0 mutation ; 0 secret exposé ; contrats read-only
API candles réelle	GET /candles?symbol=BTCUSDT&interval;=1m	Réponse source real (Binance) ; bougies conformes
E2E desktop 1440	/session, BTCUSDT puis BOOM500	Badge LIVE avec ticks réels ; SIM étiqueté ; overlays silencieux ; panel sans bouton ; 7/7 timeframes

Test	Conditions	Résultat
Coupure du flux (stale)	kill du process market-service, observation 16 s	Badge STALE à 16 s, âge du flux affiché, aucune valeur figée présentée comme vivante
Reprise (resync)	relance du market-service	Resync complet : candles et prix recalés, badge restauré, âge revenu à 0 s
Mobile 390	viewport 390 px, mesure scrollWidth	11 px de débordement détectés, corrigés (3e4d57b), re-mesure à 390 exactement
Qualité statique	tsc --noEmit ; eslint --max-warnings=0 ; console navigateur	0 erreur TypeScript, 0 warning ESLint, 0 erreur console

Tableau 4 — Campagne de tests : statique, runtime, mobile et qualité.

Le test de coupure mérite un mot, car il valide la promesse centrale de la protection stale data. En tuant le market-service, le flux de prix se fige immédiatement mais le transport côté navigateur peut rester connecté un certain temps : c'est précisément le cas que la protection doit couvrir. Huit secondes après le dernier tick (STALE_MS = 8 000), le badge bascule en STALE et l'interface affiche l'âge du flux en secondes ; à la relance du service, la resynchronisation recale les bougies et le badge revient à son état nominal sans intervention manuelle. Les captures desktop-feed-lost et desktop-stale-16s documentent les deux instants clés.

7. Points non vérifiés

Un audit honnête se termine par la liste de ce qu'il n'a pas pu vérifier. Les limites suivantes sont assumées et tracées pour une future campagne de tests ; aucune n'invalide le verdict, mais plusieurs devront être levées avant un usage réel avec des fonds.

- **Broker MT5 réel.** Non connectable depuis cet environnement de développement : le comportement avec un compte broker authentique (données de compte, positions réelles, marges, symboles réels du broker) reste à valider en conditions réelles.
- **Build de production.** L'audit a porté sur le serveur de développement ; un passage en build de production (rewrites, variables d'environnement, optimisations Next) n'a pas été exercé.
- **Charge multi-utilisateurs.** Le market-service agrège un WebSocket Binance partagé ; sa tenue de charge avec de nombreuses sessions simultanées n'a pas été mesurée.
- **Réseau dégradé prolongé.** Au-delà du test local kill/relance, les reconnexions longues (latence élevée, coupures intermittentes, incidents côté exchange) n'ont pas été simulées.
- **Montée en durée.** La stabilité du flux sur de très longues sessions (plus de 24 heures) n'a pas été observée.

- **Isolation fine avec broker réel.** La séparation par utilisateur est garantie côté API et mocks ; elle devra être re-testée une fois les credentials réels par utilisateur branchés.

8. Phase 2 (liste, sans implémentation)

Conformément à la consigne, le backlog ci-dessous est listé sans aucune implémentation. Les items sont ordonnés par priorité implicite : ce qui débloque des données réelles d'abord, ce qui améliore l'expérience ensuite, ce qui durcit la production enfin. Chacun fera l'objet, le moment venu, d'une feature dédiée avec vérification E2E et commit séparé.

1. Adaptateur broker réel (MT5 ou bridge équivalent) implémentant le contrat MarketDataProvider côté compte, avec credentials chiffrés par utilisateur et relais serveur uniquement.

2. Mapping symboles tri-couche complet (interne / broker / chart, par exemple XAUUSD contre XAUUSDm contre GOLD), avec table de correspondance et UI de résolution.

3. Persistance des candles historiques pour un démarrage instantané des graphiques et une meilleure résilience aux coupures.

4. Alertes prix / SL / TP côté client, alimentées par le flux live existant.

5. Observabilité du market-service : métriques (latence, ticks par seconde, reconnexions), logs structurés, reconnexion résiliente avec backoff.

6. Chart TradingView Lightweight Charts si la licence est confirmée, en remplacement éventuel d'ECharts.

7. Tests de charge WebSocket (sessions simultanées, plafond de providers) et durcissement production (build, environnement, supervision).

8. Comptes multi-brokers par utilisateur, avec sélection du compte actif dans la station Session Live.